

Hackathon Problem Statement: Image Classification with Duplicate Image Removal

Objective:

Participants are tasked with building an image classification model that can accurately classify images into one of 22 predefined classes. The dataset consists of 16,317 images distributed across these 22 classes. The challenge is not only to train a high-accuracy classification model but also to implement a duplicate image removal mechanism to ensure the model's performance is evaluated on a clean, deduplicated dataset.

Problem Description:

1. Dataset Overview:

- The dataset contains 16,317 images divided into 22 classes.
- Each class represents a distinct category, and the images within each class may contain duplicates or near-duplicates.

2. Task: Image Classification Model:

- Participants must train a high accuracy classification model.
- The model should predict the correct class label for each image in the test set.
- The performance of the model will be evaluated based on classification accuracy.

4. Duplicate Image Removal:

- Participants must implement a duplicate image removal mechanism to identify and remove duplicate or near-duplicate images for the test dataset.
- The mechanism should be robust enough to handle:
 - Exact duplicates (identical images).
 - Near-duplicates (Same building pictures taken from different angle or location).

5. Evaluation Criteria:

- **Accuracy:** The primary metric for evaluation will be the classification accuracy on the deduplicated test set.
- **Deduplication Effectiveness:** The quality of the duplicate removal mechanism will also be assessed and is an important criteria of judgement. Participants must ensure that the deduplication process does not remove non-duplicate images, as this could negatively impact the dataset's integrity.

Deliverables:

1.Code for Duplicate Image Removal:

- A Python script or Jupyter notebook that implements the duplicate image removal mechanism.
- The script should take the dataset as input and output a cleaned dataset with duplicates moved to different folder.

2.Trained Classification Model:

- A trained model file (e.g., .h5, .pth, .pt, .keras or any other feasible model) that can be used for inference.

3. Inference Script:

- A Python script that loads the trained model, performs inference on the deduplicated test set, and outputs the predicted class labels.
-

Rules and Guidelines:

1. Dataset Usage:

- Participants must use the provided dataset for training and testing.

2. Deduplication Requirement:

- The test set will contain duplicates, and their deduplication mechanism will be applied to the test set before evaluating accuracy.

3. Evaluation Process:

- The submissions will be evaluated based on:
 - The effectiveness of the duplicate removal mechanism.
 - The classification accuracy on the deduplicated test set.
- Participants must submit their code, model for evaluation.

4. Submission Format:

- A zip file containing:
 - Deduplication script.
 - Trained model file.
 - Inference script.
-

Judging Criteria:

1. Classification Accuracy (60% weight):

- The accuracy of the model on the deduplicated test set.

2. Deduplication Effectiveness (30% weight):

- The ability of the mechanism to correctly identify and remove duplicates without removing non-duplicate images.

3. Code Quality (10% weight):

- The clarity, efficiency, and documentation of the code.